LevelUp Frontend Deployment on Vercel

Step-by-step setup for deploying the React/Vite frontend and updating the already-deployed backend.

1. Project Summary

Your frontend is a Vite + React app inside the existing MERN project. The backend is already deployed, so Vercel only needs to build and host the client folder. Do not deploy the whole LevelUp repository as the app root; point Vercel to the frontend folder.

Setting	Value for this project
Vercel root directory	client/levelUp
Framework preset	Vite
Install command	npm install
Build command	npm run build
Output directory	dist
Frontend env key	VITE_API_URL
Google env key	VITE_GOOGLE_CLIENT_ID

2. Before You Deploy

- Confirm the backend deployed URL is working, for example: https://your-backend-domain.com/test.
- Confirm backend CORS allows the Vercel frontend domain after deployment.
- Make sure your frontend code is pushed to GitHub, GitLab, or Bitbucket.
- Keep all secrets out of Git. Add runtime values in Vercel Environment Variables.
- If Google sign-in is used, keep the same OAuth client ID in frontend and backend configuration.

3. Deploy Frontend on Vercel

1. Open https://vercel.com and sign in.
2. Click Add New, then Project.
3. Import the Git repository that contains the LevelUp project.
4. When Vercel asks for Root Directory, select client/levelUp.

5 Set Framework Preset to Vite if Vercel does not auto-detect it.

6 Keep Install Command as npm install.

7 Set Build Command to npm run build.

8 Set Output Directory to dist.

9 Add all required environment variables before clicking Deploy.

10 Click Deploy and wait until the build finishes successfully.

Environment Variables in Vercel

Variable	Example / Meaning
VITE_API_URL	Your deployed backend base URL, e.g. https://your-backend-domain.com
VITE_GOOGLE_CLIENT_ID	Google OAuth Web Client ID used by @react-oauth/google

Important: Vite exposes only variables that start with VITE_. Do not use REACT_APP_ for this project.

4. Backend CORS Setup for Vercel

After frontend deployment, copy the Vercel production URL, then add it to the backend CORS allowed origins. If the backend currently allows every origin during development, tighten it for production with your exact Vercel domain.

```
const allowedOrigins = [
  'http://localhost:5173',
  'https://your-vercel-app.vercel.app',
];

app.use(cors({
  origin: allowedOrigins,
  credentials: true,
}));
```

If you use cookie-based login across different domains, the backend cookie settings also matter. In production, use secure: true and sameSite: 'none' only when your frontend and backend are on different domains and HTTPS is enabled.

```
res.cookie('token', token, {
  httpOnly: true,
  secure: process.env.NODE_ENV === 'production',
  sameSite: process.env.NODE_ENV === 'production' ? 'none' : 'lax',
  path: '/',
});
```

5. Google Sign-In Setup

1. Open Google Cloud Console.
2. Go to APIs & Services, then Credentials.
3. Open your OAuth 2.0 Web Client ID.
4. Add http://localhost:5173 to Authorized JavaScript origins for local testing.
5. Add your Vercel URL, for example https://your-vercel-app.vercel.app, to Authorized JavaScript origins.
6. Save the changes.
7. Use the same client ID in Vercel as VITE_GOOGLE_CLIENT_ID.
8. Use the same client ID in the backend deployment as GOOGLE_CLIENT_ID.

6. Test After Deployment

- Open the Vercel URL in a fresh browser session.
- Test normal signup with username, email, and password.
- Test login and confirm the dashboard opens.
- Test forgot password: request token, reset password, then login with the new password.

- Test Google sign-in after OAuth origins are updated.
- Open Profile and Dashboard pages to confirm authenticated API calls work.
- Create a battle link and verify the frontend can communicate with the deployed backend.
- Check Vercel build logs only for frontend build issues; backend runtime logs are on the backend host.

7. Common Errors and Fixes

Problem	Most likely fix
Blank page on Vercel	Check Root Directory is client/levelUp and Output Directory is dist.
API calls go to wrong URL	Set VITE_API_URL in Vercel and redeploy.
Login works locally but not on Vercel	Update backend CORS and cookie sameSite/secure settings.
Google button fails	Add Vercel URL to Google OAuth Authorized JavaScript origins.
Environment variable not found	Use VITE_ prefix and redeploy after adding the variable.
404 on refresh	Vercel usually handles Vite SPA routing; add a rewrite to /index.html if needed.

Optional vercel.json for SPA refresh

Create this file inside client/levelUp only if refresh routes like /dashboard or /signin show 404.

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

8. How to Add New Backend Features to the Already-Deployed Backend

When you add a new backend feature, the deployed backend will not update automatically unless your hosting platform is connected to Git auto-deploy or you manually redeploy it. Follow this workflow.

1. Add the backend code locally: model, controller, route, middleware, or service as needed.
2. Register the route in server/src/app.js or the correct route file.
3. Add any new environment variables to the backend hosting platform dashboard.
4. Run backend tests locally: npm.cmd test from LevelUp/server.
5. Commit and push the backend changes to the branch used by your backend host.
6. Open your backend hosting platform and trigger redeploy if auto-deploy is not enabled.
7. Check deployment logs for install, build, startup, MongoDB, and env errors.
8. Test the new API endpoint using Postman, Thunder Client, or the frontend.

9 If the frontend calls this new API, update frontend service files and redeploy Vercel too.

> **Rule:** Backend feature added means backend redeploy is required. Frontend redeploy is required only when frontend code or frontend environment variables changed.

9. Final Deployment Checklist

- Frontend root directory on Vercel is client/levelUp.
- VITE_API_URL points to deployed backend, not localhost.
- VITE_GOOGLE_CLIENT_ID is set in Vercel.
- Backend GOOGLE_CLIENT_ID is set on backend host.
- Backend CORS includes the Vercel production URL.
- Cookie settings are production-ready if using cross-domain cookies.
- Signup, login, forgot password, reset password, Google sign-in, dashboard, and profile are tested after deploy.